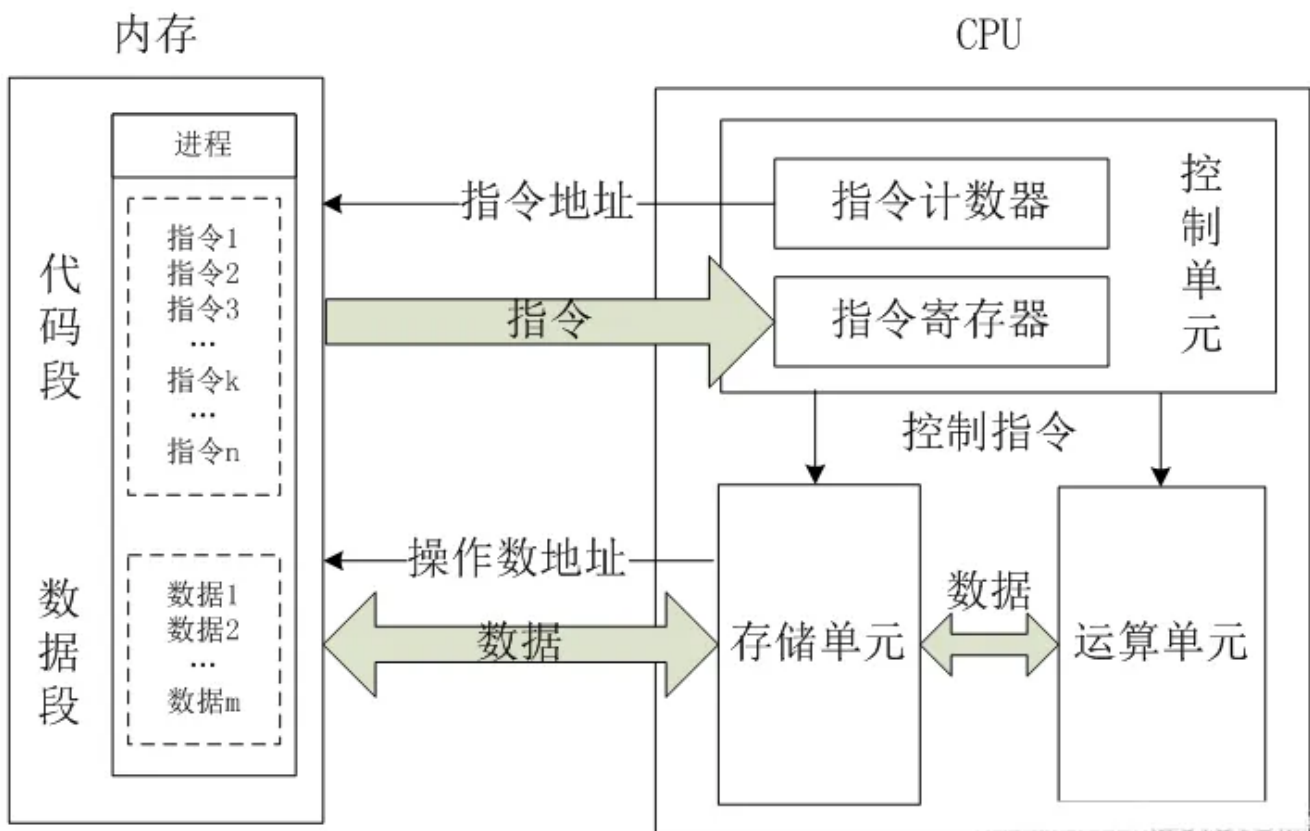


C语言内存管理专题

1. 计算机程序运行机制简介

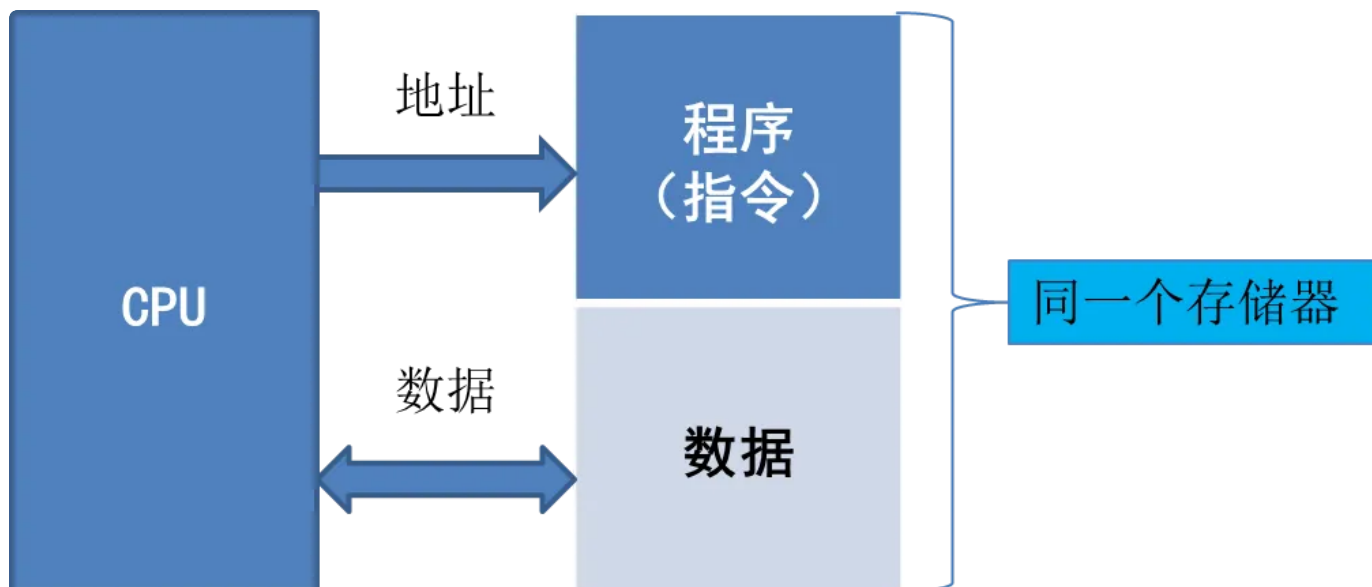
- CPU主要对指令进行解析、执行，受限于芯片体积、功耗、成本因素，CPU内部很难做到大存储，这样CPU要执行的指令和数据必须外置
- 内存芯片，往往提供地址总线 and 数据总线，方便和CPU直连

1.1 CPU和内存的关系



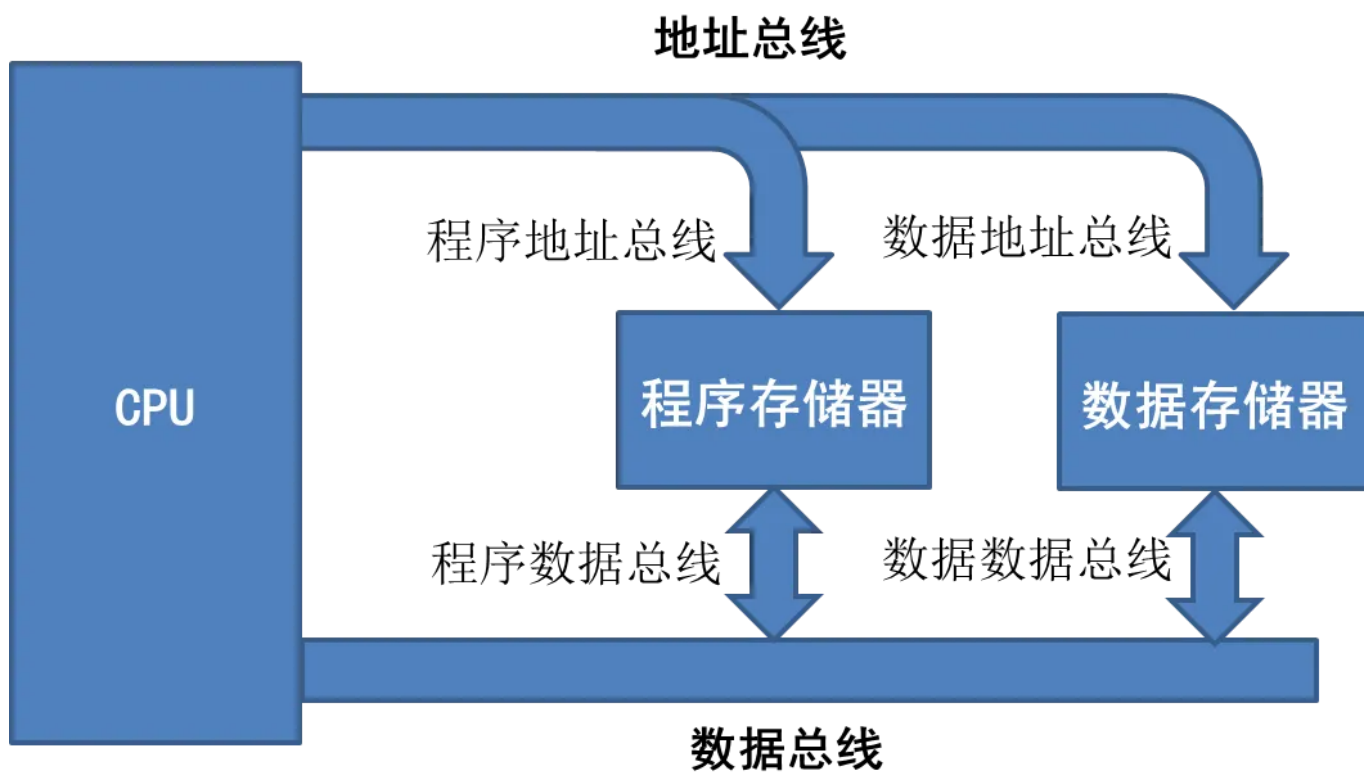
1.2 两大体系结构

- 冯诺依曼结构



冯诺依曼结构

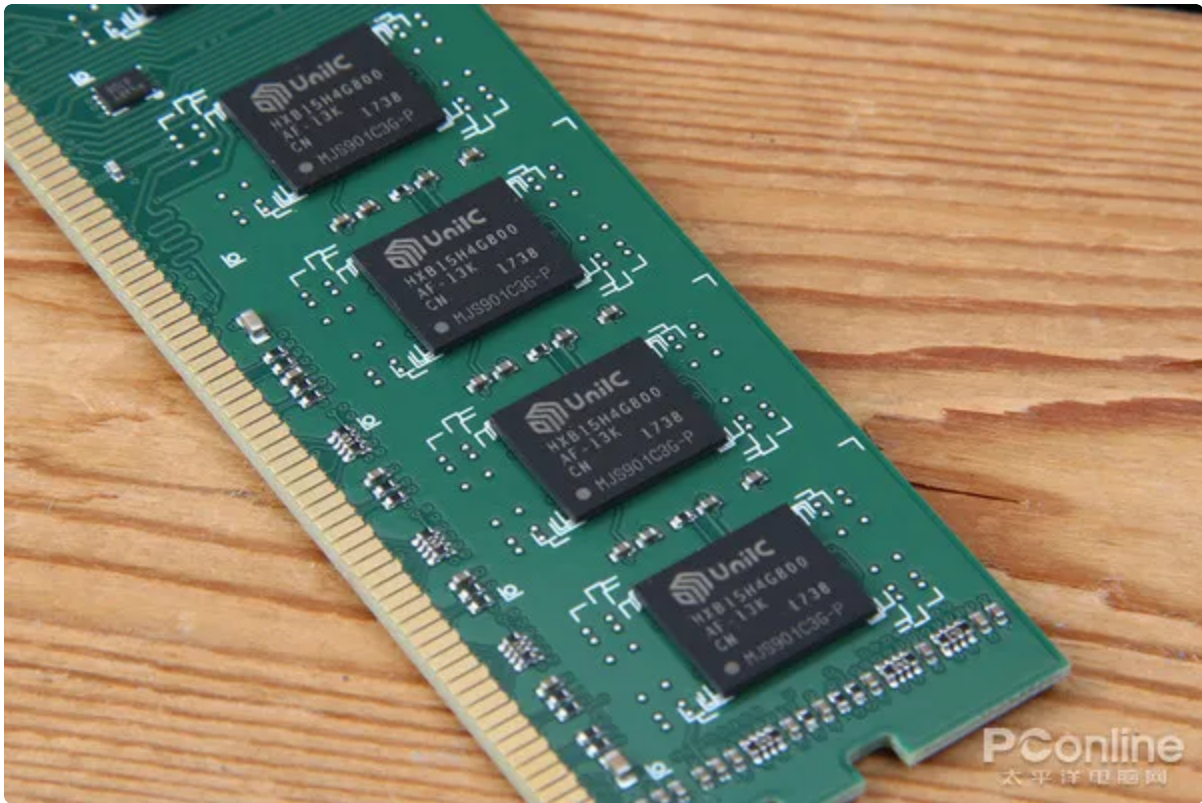
- 哈佛结构



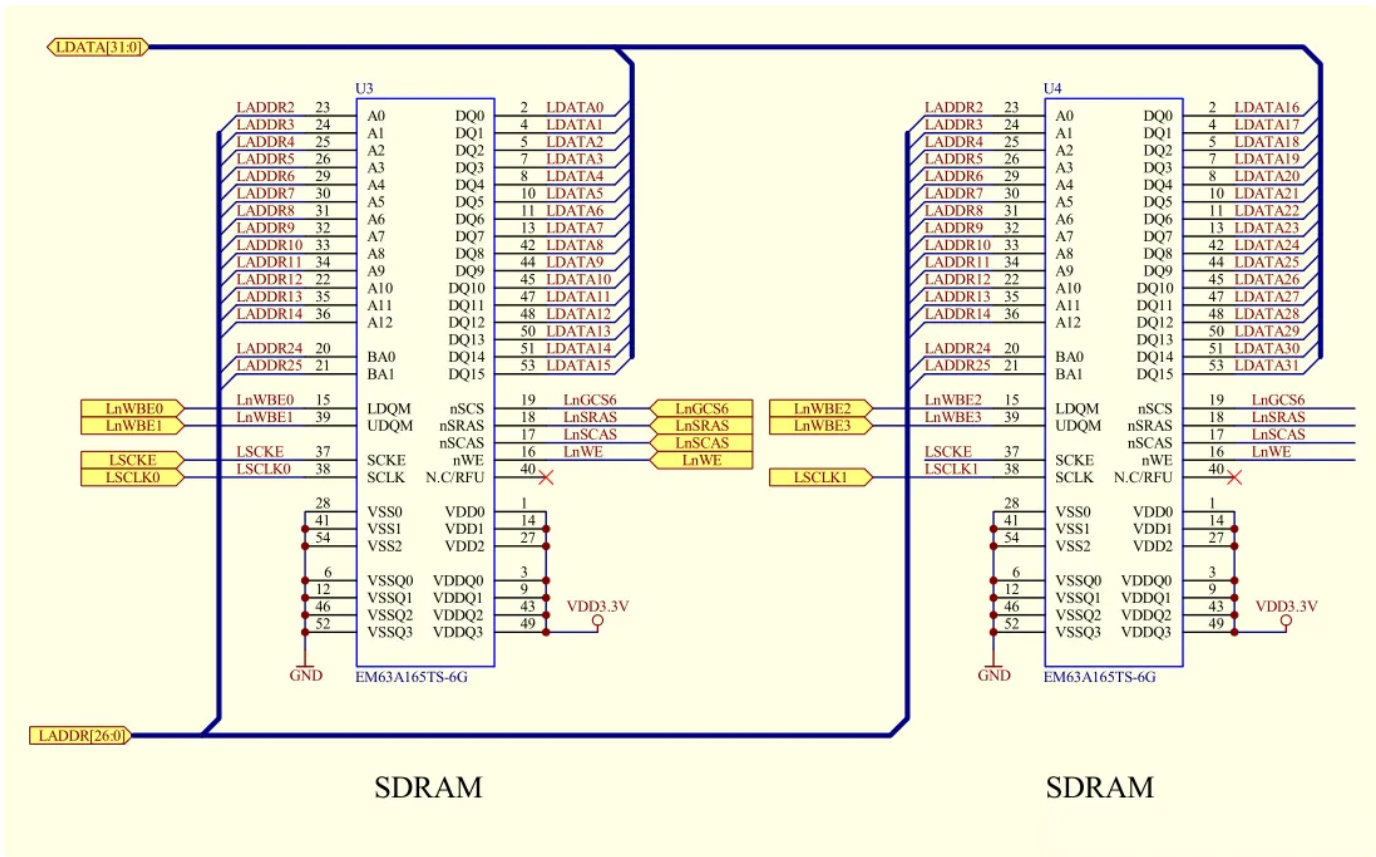
改进型哈佛结构

1.3 内存物理形态和硬件原理图

- 物理形态



- 硬件原理图



2. C语言内存空间访问方式

2.1 变量名的本质

- 对一段内存空间，方便程序员描述，让编译器可以进行正确映射关系的名称
- 编译器帮程序员，将这段空间访问的代码，翻译成正确的CPU指令
- 变量名的注意事项：
 - 编译器能够正确识别 => 有效标识符
 - 不能有重复的变量名 => 一个变量名代表一个专属空间
 - 变量名必须有类型 => 这样空间才有约束
 - 变量名有作用域 => 申请空间和释放空间

2.2 指针的本质

- 保存地址的容器，C语言的地址和计算机的地址，在定义时发生了改变
- C语言地址如何映射到计算机底层的地址：
 - C语言地址定义
 - C语言只定义了地址的形态，具体怎么用，全部交由程序员来处理
 - 不管越界访问
 - 不管有没有权限访问

```
1 int main() {
2     int *p1 = (int *)0x100;
3     char *p2 = (char *)0x100;
4     char p3[5];
5     char *p4[5];
6     char (*p5)[5] = (char (*)[5])0x100;
7     char p6[6][5];
8
9     printf("the p1: %p, p1 + 1: %p\n", p1, p1 + 1);
10    printf("the p2: %p, p2 + 1: %p\n", p2, p2 + 1);
11    printf("the p3: %p, p3 + 1: %p\n", p3, p3 + 1);
12    printf("the p4: %p, p4 + 1: %p\n", p4, p4 + 1);
13    printf("the p5: %p, p5 + 1: %p\n", p5, p5 + 1);
14    printf("the p6: %p, p6 + 1: %p\n", p6, p6 + 1);
15
16    return 0;
17 }
```

- C语言定义地址的访问：
 - `*`、`[]`
- 多维指针的逻辑模型

3. C语言内存空间结构化分配

3.1 一维数组空间及访问

- 一维数组的编译器行为（语法）
 - C语言只管分配，不管越界，因为在C语言看来，内存就是连续的，访问的权限应该交给程序员，而不是编译器。（任性）
 - 分配关心的是个数和每个元素的大小
 - 数组名不是变量名，只是为了标识这个空间，引入的常量标签（C语言压根就没有数组变量的概念）
- 特殊的一维数组
 - 字符串和字符数组
 - 重要的字符串空间处理函数
 - `strlen/sizeof`
 - `strcpy/strncpy`

- memcpy/memset : 单位 B
- sprintf/snprintf
- printf的%s含义

3.2 二维及多维数组空间及访问

- 二维数组的内存模型
 - 计算机内存是线性编址，一维空间如何描述人类社会的二维空间
 - 二维数组名的地址本质
- 三维或多维数组的内存模型
- **main函数参数空间运行分析**
 - char **args 和 char buf[4][5]的区别

3.3 结构体空间和数组空间

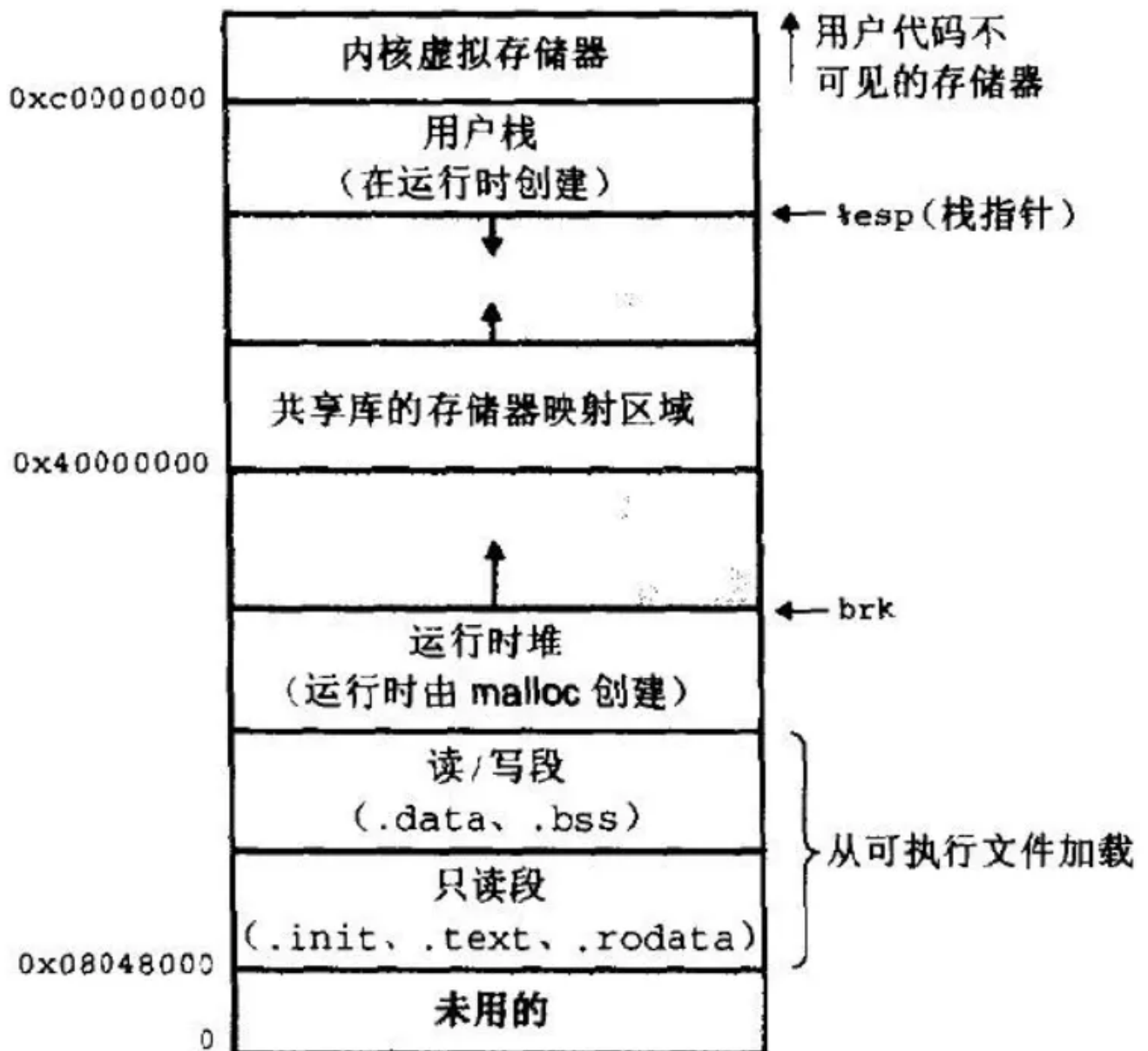
- 区别和应用场景
- 语法说明

4. C语言内存分段管理及权限管理

4.1 一个经典的错误

```
1 int main() {  
2     char *str = "2021";  
3     str[3] = '2';  
4     printf("str=%s\n", str);  
5     return 0;  
6 }
```

4.2 内存分段管理模型和权限



○ 各段使用的注意事项

- 低地址段和3G空间以上的段，不可读，不可写，被操作系统保护
- 栈空间由编译器编译时，保证内存资源的释放和申请
- 堆空间必须由程序员来维护（内存泄漏的重灾区）

4.3 内存各段空间分配和使用

- 代码段.text
- 只读数据段.rodata
- 数据段.data

- static区
- 全局区
- 堆段
- 栈段